

Docker Tutorial - Comprehensive Technical Reference Guide

Table of Contents

1. Introduction to Docker
 2. Docker Installation
 3. Basic Docker Commands
 4. Advanced Docker Run Options
 5. Docker Images
 6. CMD vs ENTRYPOINT
 7. Docker Networking
 8. Docker Storage
 9. Docker Compose
 10. Docker Registry
 11. Docker Engine Architecture
 12. Docker on Windows and Mac
 13. Container Orchestration
-

Introduction to Docker

What Problem Does Docker Solve?

Concept: Docker addresses the “Matrix from Hell” - compatibility issues between different application components, their dependencies, and the underlying operating system.

The Problem Scenario: - Setting up an end-to-end application stack with multiple technologies (Node.js, MongoDB, Redis, Ansible) - Compatibility issues with OS versions - Dependency conflicts (one service needs library v1, another needs library v2) - Architecture changes requiring compatibility re-checks - Difficult developer onboarding with complex setup instructions - Environment differences between development, test, and production

The Docker Solution: - Each component runs in a separate container with its own dependencies - All containers share the same VM and OS but have isolated environments - One-time Docker configuration, simple `docker run` command for all developers - Consistent behavior across all environments

Containers vs Virtual Machines

Concept: Containers and VMs both provide isolation but work differently at the infrastructure level.

Container Architecture:

Application + Dependencies

```

|-- Container Layer (writable)
|-- Docker Image Layers (read-only)
|-- Docker Engine
|-- Operating System
-- Hardware Infrastructure

```

Virtual Machine Architecture:

```

Application + Dependencies
|-- Guest Operating System (full OS)
|-- Hypervisor (ESX, Hyper-V)
|-- Host Operating System
-- Hardware Infrastructure

```

Key Differences:

Aspect	Containers	Virtual Machines
Size	Megabytes	Gigabytes
Startup Time	Seconds	Minutes
Resource Usage	Lightweight, shared kernel	Heavy, full OS per VM
Isolation	Process-level, shared kernel	Complete isolation, separate kernels
OS Compatibility	Must match host kernel (Linux containers on Linux)	Can run different OS types

Important Note: Containers share the underlying OS kernel. This means: - Linux containers require a Linux kernel - Windows containers require a Windows kernel - You cannot run Windows containers on a Linux host without a Windows VM

What are Containers?

Concept: Containers are completely isolated environments with their own processes, network interfaces, and mounts, but they all share the same OS kernel.

Container Types: - LXC (Linux Containers) - LXD - LXCFS

Docker's Role: Docker utilizes LXC containers but provides a high-level interface with powerful functionalities, making containers easy to use.

Operating System Architecture

Concept: Understanding how operating systems are structured helps explain how Docker works.

OS Components: 1. **OS Kernel** - Interacts with underlying hardware (e.g., Linux kernel) 2. **Software Layer** - UI, drivers, compilers, file managers, developer tools

Different Distributions: - Ubuntu, Fedora, SUSE, CentOS all use the Linux kernel - The difference is in the software layer above the kernel

Docker and Kernels:

Docker host with Ubuntu can run containers based on:

```
docker run ubuntu
docker run debian
docker run fedora
docker run centos
```

All these work because they share the Linux kernel. The Docker container only includes the additional software that differentiates the distributions.

Limitation:

This will NOT work on a Linux Docker host:

```
docker run windows-server
```

Windows containers require a Windows kernel and must run on Windows Server with Docker.

Docker Installation

Installation Methods

Concept: Docker is available in two editions - Community Edition (free) and Enterprise Edition (paid with additional features).

Docker Editions

Community Edition (CE): - Free Docker products - Available on Linux, Mac, Windows, AWS, Azure

Enterprise Edition (EE): - Certified and supported container platform - Enterprise add-ons: - Image management - Image security - Universal Control Plane - Container runtime orchestration

Installation on Linux (Ubuntu)

Prerequisites: - 64-bit system - Supported Ubuntu versions: Cosmic, Bionic, or Xenial

Step 1: Check Ubuntu Version

```
cat /etc/lsb-release
```

Breakdown: - Verifies your Ubuntu version is supported

Step 2: Check for Old Docker Versions

```
sudo apt-get remove docker docker-engine docker.io containerd runc
```

Breakdown: - Removes any existing older Docker installations - Ensures clean installation

Step 3: Installation via Convenience Script (Recommended)

```
# Download the installation script
```

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
# Execute the script
```

```
sudo sh get-docker.sh
```

Breakdown: - curl -fsSL: Downloads the script - -f: Fail silently on HTTP errors - -s: Silent mode - -S: Show errors - -L: Follow redirects - Automates the entire installation process - Works on most operating systems

Alternative Method: Manual Repository Setup

```
# Update package index
```

```
sudo apt-get update
```

```
# Install prerequisites
```

```
sudo apt-get install \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg-agent \
    software-properties-common
```

```
# Add Docker's official GPG key
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

```
# Set up stable repository
```

```
sudo add-apt-repository \
    "deb [arch=amd64] https://download.docker.com/linux/ubuntu \
    $(lsb_release -cs) \
    stable"
```

```
# Install Docker Engine
```

```
sudo apt-get update
```

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

Step 4: Verify Installation

```
docker version
```

Expected Output:

```
Client: Docker Engine - Community
Version:      19.03.1
API version:  1.40
...

Server: Docker Engine - Community
Engine:
Version:      19.03.1
API version:  1.40 (minimum version 1.12)
...
```

Step 5: Test Docker Installation

```
sudo docker run docker/whalesay cowsay "Hello World"
```

Breakdown: - `docker run`: Runs a container - `docker/whalesay`: The image name - `cowsay "Hello World"`: Command to execute inside the container - Docker will pull the image from Docker Hub if not present locally - Displays an ASCII art whale saying “Hello World”

Installation Location and File Structure

Concept: Docker stores all its data in a specific directory structure on the host.

Default Installation Directory:

```
/var/lib/docker/
```

Directory Structure:

```
/var/lib/docker/
|-- aufs/
|-- containers/
|-- image/
|-- volumes/
-- [other directories]
```

Purpose of Each Directory: - `containers/`: Stores files related to running containers - `image/`: Stores image files - `volumes/`: Stores volumes created by containers - `aufs/` (or other storage driver directories): Storage driver specific files

Basic Docker Commands

Docker Run Command

Concept: The `docker run` command is used to create and start a container from an image.

Basic Usage:

```
docker run nginx
```

Breakdown: - `docker run`: Command to run a container - `nginx`: Image name

- **First Execution:** Docker pulls the image from Docker Hub - **Subsequent**

Executions: Uses cached local image

What Happens: 1. Docker checks if image exists locally 2. If not found, pulls from Docker Hub (`docker pull nginx`) 3. Creates a container from the image 4. Starts the container

Listing Containers

Command 1: List Running Containers

```
docker ps
```

Breakdown: - Lists all currently running containers - Shows: Container ID, Image, Command, Created time, Status, Ports, Name

Example Output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAME
a043d2346f12	nginx	"nginx -g 'daemon of...'"	2 minutes ago	Up 2 minutes	80/tcp	silly_sammet

Command 2: List All Containers (Including Stopped)

```
docker ps -a
```

Breakdown: - `-a` or `--all`: Shows all containers (running and stopped/exited)

- Useful for seeing containers that have completed execution

Stopping Containers

Concept: Gracefully stops a running container.

Command:

```
docker stop <container_id_or_name>
```

Examples:

```
# Using container ID
```

```
docker stop a043d2346f12
```

```
# Using container name
```

```
docker stop silly_sammet
```

```
# Using partial container ID (must be unique)
```

```
docker stop a04
```

Breakdown: - Sends SIGTERM signal to the main process - Container status changes to “Exited” - Container still exists but is stopped

Verification:

Won't show in running containers
docker ps

Will show in all containers
docker ps -a

Removing Containers

Concept: Permanently deletes a stopped container.

Command:

```
docker rm <container_id_or_name>
```

Example:

```
docker rm silly_sammet
```

Breakdown: - Removes the container permanently - Container must be stopped first - Frees up disk space - If successful, prints the container name

Verification:

```
docker ps -a  
# Container should no longer appear in the list
```

Managing Images

Command 1: List Images

```
docker images
```

Example Output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	5a3221f0137b	2 weeks ago	142MB
redis	latest	7614ae9453d1	3 weeks ago	113MB
ubuntu	latest	9140108b62dc	4 weeks ago	72.9MB
alpine	latest	e7d92cdc71fe	5 weeks ago	5.59MB

Breakdown: - REPOSITORY: Image name - TAG: Version tag (default: latest)
- IMAGE ID: Unique identifier - CREATED: When the image was created - SIZE: Image size

Command 2: Remove an Image

```
docker rmi <image_name>
```

Example:

```
docker rmi nginx
```

Breakdown: - rmi: Remove image - **Requirement:** No containers (running or stopped) must be using this image - Must delete all dependent containers first

If Image is in Use:

Error message

Error response from daemon: conflict: unable to remove repository reference "nginx" (must first

Solution: Remove container first

```
docker rm <container_id>
```

```
docker rmi nginx
```

Pulling Images

Concept: Downloads an image from Docker registry without running it.

Command:

```
docker pull <image_name>
```

Example:

```
docker pull ubuntu
```

Breakdown: - Downloads image to local system - Stores in `/var/lib/docker/images/`
- Subsequent `docker run` commands won't need to download - Useful for pre-fetching images

Execute Commands in Running Containers

Concept: Run commands inside an already running container.

Command:

```
docker exec <container_id_or_name> <command>
```

Example:

```
docker exec silly_sammet cat /etc/hosts
```

Breakdown: - `docker exec`: Execute command in running container -
`silly_sammet`: Container name or ID - `cat /etc/hosts`: Command to execute

Use Cases: - Debugging - Checking files inside container - Running administrative commands

Container Lifecycle

Concept: Containers exit when their main process completes.

Example with Ubuntu:


```
docker run ubuntu
```

What Happens: 1. Docker creates container from Ubuntu image 2. Starts the container 3. Ubuntu's default command (**bash**) runs 4. Bash finds no terminal (not in interactive mode) 5. Bash exits immediately 6. Container stops

Verification:

```
docker ps
# Container won't be listed (not running)
```

```
docker ps -a
# Container shown with "Exited" status
```

Why Does This Happen? - Containers are NOT meant to host operating systems - Containers run a specific task/process - Container lives only as long as the process inside it - When process stops/crashes, container exits

Running Container with Custom Command

Concept: Override the default command to keep container running.

Command:

```
docker run ubuntu sleep 5
```

Breakdown: - ubuntu: Base image - sleep 5: Command to execute (overrides default bash) - Container runs sleep command for 5 seconds - After 5 seconds, sleep exits, container stops

Verification:

```
# Immediately after running
docker ps
# Shows container running

# After 5 seconds
docker ps
# Container no longer listed (exited)

docker ps -a
# Shows container with "Exited" status
```

Advanced Docker Run Options

Attach vs Detach Mode

Concept: Control whether your terminal is attached to the container's console.

Attached Mode (Default):

```
docker run kodekloud/simple-webapp
```

Breakdown: - Container runs in foreground - Terminal shows container output (logs) - Terminal is blocked - cannot execute other commands - CTRL+C stops the container

Example Output:

```
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
```

Detached Mode:

```
docker run -d kodekloud/simple-webapp
```

Breakdown: - `-d` or `--detach`: Runs container in background - Returns container ID and returns to prompt immediately - Container continues running in background

Example Output:

```
a043d2346f124d8e8a9f7c3b1d2e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e
```

Re-attaching to Detached Container:

```
docker attach <container_id_or_name>
```

Example:

```
docker attach a043d
```

Breakdown: - Attaches terminal to running container - Shows live output from container - Can use partial container ID (first few characters)

Interactive Mode

Concept: Some applications require user input. Use interactive mode to provide input to containerized applications.

Problem Scenario:

```
# Application that prompts for input
docker run kodekloud/simple-prompt-docker
```

Output:

```
Welcome! Please enter your name:
```

Issue: Container doesn't wait for input and exits immediately.

Solution - Interactive Mode:

```
docker run -i kodekloud/simple-prompt-docker
```

Breakdown: - `-i` or `--interactive`: Keeps STDIN open - Container can now accept input - But prompt is not displayed

Example:

```
docker run -i kodekloud/simple-prompt-docker
# Type your name (no prompt shown)
John
# Output: Hello John, Welcome!
```

Complete Interactive Mode with Terminal:

```
docker run -it kodekloud/simple-prompt-docker
```

Breakdown: - `-i`: Interactive mode (STDIN) - `-t`: Pseudo-TTY (terminal) - Together: `-it` provides full interactive terminal - Prompt is displayed properly

Example:

```
docker run -it kodekloud/simple-prompt-docker
```

Output:

```
Welcome! Please enter your name: John
Hello John, Welcome!
```

Port Mapping

Concept: Map container ports to host ports to access containerized applications from outside.

Internal IP Access:

```
# Container internal IP (only accessible within Docker host)
docker inspect <container_id> | grep IPAddress
# Output: "IPAddress": "172.17.0.2"

# Access from within Docker host
curl http://172.17.0.2:5000
```

Problem: Internal IP not accessible from outside Docker host.

Solution - Port Mapping:

```
docker run -p 80:5000 kodekloud/simple-webapp
```

Breakdown: - `-p` or `--publish`: Publish container port to host - Format: `-p <host_port>:<container_port>` - 80: Port on Docker host - 5000: Port inside container - Traffic to host:80 → forwarded to → container:5000

Access Pattern:

```
# From external systems
curl http://192.168.1.5:80
# Where 192.168.1.5 is the Docker host IP
```

Multiple Port Mappings:

```
# Run multiple instances on different ports
docker run -p 5000:5000 kodekloud/simple-webapp
docker run -p 5001:5000 kodekloud/simple-webapp
docker run -p 5002:5000 kodekloud/simple-webapp
```

Database Example:

```
# MySQL instance 1
docker run -p 3306:3306 mysql
```

```
# MySQL instance 2
docker run -p 8306:3306 mysql
```

Breakdown: - Same container port (3306) mapped to different host ports -
Cannot map same host port twice - Each instance accessible on different port

Volume Mapping

Concept: Persist data outside containers so it survives container deletion.

Problem Scenario:

```
docker run mysql
```

Issues: - Data stored in `/var/lib/mysql` inside container - Container deletion
= data loss - Data not persistent

Solution - Volume Mapping:

```
docker run -v /opt/datadir:/var/lib/mysql mysql
```

Breakdown: - `-v` or `--volume`: Mount a volume - Format: `-v <host_path>:<container_path>`
- `/opt/datadir`: Directory on Docker host - `/var/lib/mysql`: Directory inside
container - Data written to container path → actually stored on host path

What Happens: 1. Container writes data to `/var/lib/mysql` 2. Docker maps
writes to `/opt/datadir` on host 3. Data persists even after container deletion
4. New container can use same volume

Example:

```
# Create directory on host
mkdir -p /opt/datadir
```

```
# Run MySQL with volume mapping
docker run -v /opt/datadir:/var/lib/mysql mysql
```

```
# Data remains in /opt/datadir even after:
docker rm <mysql_container>
```

Inspecting Containers

Concept: Get detailed information about a container in JSON format.

Command:

```
docker inspect <container_id_or_name>
```

Example:

```
docker inspect silly_sammet
```

Output (Sample):

```
[
  {
    "Id": "a043d2346f124d8e8a9f7c3b1d2e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e",
    "Created": "2024-12-01T10:30:00.000000000Z",
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 12345,
      "ExitCode": 0
    },
    "Config": {
      "Hostname": "a043d2346f12",
      "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
      ],
      "Cmd": ["nginx", "-g", "daemon off;"]
    },
    "NetworkSettings": {
      "IPAddress": "172.17.0.2",
      "Ports": {
        "80/tcp": [{"HostIp": "0.0.0.0", "HostPort": "8080"}]
      }
    },
    "Mounts": [
      {
        "Source": "/opt/datadir",
        "Destination": "/var/lib/mysql",
        "Mode": "",

```

```

    "RW": true
  }
]
]

```

Breakdown: - Returns complete container configuration - Includes state, network settings, mounts, environment variables - Useful for debugging and troubleshooting

Viewing Container Logs

Concept: View logs (stdout) from a container running in detached mode.

Command:

```
docker logs <container_id_or_name>
```

Example:

```
docker run -d kodekloud/simple-webapp
docker logs a043d
```

Output:

```

* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
192.168.1.100 - - [01/Dec/2024 10:30:00] "GET / HTTP/1.1" 200 -
192.168.1.101 - - [01/Dec/2024 10:30:05] "GET /api HTTP/1.1" 200 -

```

Breakdown: - Shows all output written to STDOUT by container - Equivalent to terminal output in attached mode - Useful for debugging detached containers

Docker Images

Understanding Image Tags

Concept: Tags specify versions of Docker images.

Image Naming Convention:

```
<username>/<repository>:<tag>
```

Examples:

```
# Using specific version
docker run redis:5.0
```

```
# Using default (latest)
docker run redis
```

Equivalent to:

```
docker run redis:latest
```

Breakdown: - **redis:** Repository name - **5.0:** Tag (version) - **latest:** Default tag if none specified

Finding Available Tags: 1. Visit Docker Hub: <https://hub.docker.com> 2. Search for image (e.g., “redis”) 3. Check “Tags” tab 4. Lists all available versions with their tags

Example Tags for Redis: - **redis:7.0.5** - **redis:7.0** - **redis:7** - **redis:latest** (points to newest version) - **redis:6.2.7** - **redis:6.2** - **redis:alpine** (minimal Alpine Linux based)

Creating Custom Images

Concept: Build your own Docker images using a Dockerfile.

Scenario: Containerizing a Flask web application

Manual Setup Steps (What we’re automating): 1. Start with Ubuntu OS 2. Update apt repositories 3. Install dependencies using apt 4. Install Python dependencies using pip 5. Copy source code to /opt 6. Run web server using Flask command

Dockerfile Basics

Concept: Dockerfile contains instructions to build a Docker image.

Dockerfile Format:

```
INSTRUCTION arguments
```

All Instructions (Uppercase): - FROM - RUN - COPY - ADD - WORKDIR - ENV - EXPOSE - CMD - ENTRYPOINT

Building a Flask Application Image

Dockerfile:

```
FROM ubuntu
```

```
RUN apt-get update
```

```
RUN apt-get install -y python python-pip
```

```
RUN pip install flask
```

```
COPY app.py /opt/app.py
```

```
ENTRYPOINT FLASK_APP=/opt/app.py flask run --host=0.0.0.0
```

Instruction Breakdown:

1. FROM - Base Image

```
FROM ubuntu
```

- **Purpose:** Defines the base OS for the container
- Every Dockerfile MUST start with FROM
- Uses official Ubuntu image from Docker Hub
- Format: FROM <image> or FROM <image>:<tag>

2. RUN - Execute Commands

```
RUN apt-get update
```

```
RUN apt-get install -y python python-pip
```

```
RUN pip install flask
```

- **Purpose:** Executes commands during image build
- Runs on the base image
- Each RUN creates a new layer
- -y flag: Auto-confirms installations

3. COPY - Copy Files

```
COPY app.py /opt/app.py
```

- **Purpose:** Copies files from host to image
- Format: COPY <source> <destination>
- app.py: File on local system (same directory as Dockerfile)
- /opt/app.py: Destination path inside image

4. ENTRYPOINT - Startup Command

```
ENTRYPOINT FLASK_APP=/opt/app.py flask run --host=0.0.0.0
```

- **Purpose:** Command to run when container starts
- Sets environment variable and starts Flask server
- --host=0.0.0.0: Makes server accessible externally

Building the Image

Command:

```
docker build -t mmumshad/my-custom-app .
```

Breakdown: - docker build: Build image from Dockerfile - -t: Tag the image
- mmumshad/my-custom-app: Tag format <username>/<image-name> - .: Build context (current directory, where Dockerfile is located)

Build Process Output:

```
Sending build context to Docker daemon 3.072kB
Step 1/6 : FROM ubuntu
--> 9140108b62dc
```



```

Step 2/6 : RUN apt-get update
----> Running in a043d2346f12
... apt update output ...
----> 7f3a1b2c3d4e
Step 3/6 : RUN apt-get install -y python python-pip
----> Running in 5e6f7a8b9c0d
... installation output ...
----> 1a2b3c4d5e6f
Step 4/6 : RUN pip install flask
----> Running in 7g8h9i0j1k2l
... pip install output ...
----> 2b3c4d5e6f7a
Step 5/6 : COPY app.py /opt/app.py
----> 3c4d5e6f7a8b
Step 6/6 : ENTRYPOINT FLASK_APP=/opt/app.py flask run --host=0.0.0.0
----> Running in 4d5e6f7a8b9c
----> 5e6f7a8b9c0d
Successfully built 5e6f7a8b9c0d
Successfully tagged mmumshad/my-custom-app:latest

```

Layered Architecture

Concept: Docker builds images in layers, each instruction creating a new layer.

Layer Structure:

Layer 6: ENTRYPOINT [Flask command]	(~1KB)
Layer 5: Source Code (app.py)	(~10KB)
Layer 4: Python packages (Flask)	(~50MB)
Layer 3: Python + pip installation	(~300MB)
Layer 2: apt-get update	(~10MB)
Layer 1: Base Ubuntu OS	(~120MB)

Viewing Layer History:

```
docker history mmumshad/my-custom-app
```

Output:

IMAGE	CREATED	CREATED BY	SIZE
5e6f7a8b9c0d	2 mins ago	ENTRYPOINT FLASK_APP=/opt/app.py flask run	0B
3c4d5e6f7a8b	2 mins ago	COPY app.py /opt/app.py	1.2KB
2b3c4d5e6f7a	3 mins ago	RUN pip install flask	45MB
1a2b3c4d5e6f	5 mins ago	RUN apt-get install -y python python-pip	289MB
7f3a1b2c3d4e	8 mins ago	RUN apt-get update	12MB
9140108b62dc	4 weeks ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	120MB

Docker Build Cache

Concept: Docker caches layers to speed up rebuilds.

How Cache Works:

Initial Build:

```
docker build -t my-app .
```

All layers built from scratch.

Subsequent Build (No changes):

```
docker build -t my-app .
```

Output shows:

```
Step 1/6 : FROM ubuntu
---> Using cache
Step 2/6 : RUN apt-get update
---> Using cache
Step 3/6 : RUN apt-get install -y python python-pip
---> Using cache
...
```

Build with Failure:

```
Step 1/6 : FROM ubuntu
---> Using cache
Step 2/6 : RUN apt-get update
---> Using cache
Step 3/6 : RUN apt-get install -y python python-pip
---> ERROR: Network timeout
```

Fixing and Rebuilding:

```
# Fix network issue, rebuild
docker build -t my-app .
```

Output:

```
Step 1/6 : FROM ubuntu
---> Using cache
Step 2/6 : RUN apt-get update
---> Using cache
Step 3/6 : RUN apt-get install -y python python-pip
---> Running in ... (rebuilds from failed step)
```

Benefits of Caching: - Faster rebuilds - Only rebuilds changed layers and layers after - Efficient development workflow

Example with Code Changes:

```
# Optimized Dockerfile
FROM ubuntu
RUN apt-get update
RUN apt-get install -y python python-pip
RUN pip install flask
COPY app.py /opt/app.py
ENTRYPOINT FLASK_APP=/opt/app.py flask run
```

When only app.py changes: - Layers 1-4: Cached - Layer 5: Rebuilt (COPY command) - Layer 6: Rebuilt (depends on layer 5)

Publishing Images to Docker Hub

Step 1: Tag the Image

```
docker build -t mmumshad/my-custom-app .
```

or

```
docker image tag my-custom-app mmumshad/my-custom-app
```

Step 2: Push to Docker Hub

```
docker push mmumshad/my-custom-app
```

Breakdown: - Image must be tagged with your Docker Hub username - Format: <dockerhub_username>/<repository>:<tag> - Must be logged in: docker login

Environment Variables

Concept: Pass configuration to containers without modifying code.

Application Code Example (Python Flask):

```
# Original hardcoded version
app_color = "red"

# Updated to use environment variable
import os
app_color = os.environ.get('APP_COLOR')
```

Running Without Environment Variable:

```
docker run simple-webapp-color
# Uses default or may fail if no default
```

Running With Environment Variable:

```
docker run -e APP_COLOR=blue simple-webapp-color
```

Breakdown: - -e or --env: Set environment variable - Format: -e KEY=VALUE
- Application reads APP_COLOR from environment

Multiple Environment Variables:

```
docker run -e APP_COLOR=blue -e DB_HOST=mysql -e DB_PORT=3306 simple-webapp-color
```

Inspecting Environment Variables:

```
docker inspect <container_name>
```

Output (relevant section):

```
"Config": {  
  "Env": [  
    "APP_COLOR=blue",  
    "DB_HOST=mysql",  
    "DB_PORT=3306",  
    "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"  
  ]  
}
```

CMD vs ENTRYPOINT

Understanding CMD

Concept: CMD specifies the default command to run when container starts.

Example Dockerfiles:

NGINX Dockerfile:

```
FROM ubuntu  
RUN apt-get update  
RUN apt-get install -y nginx  
CMD ["nginx"]
```

MySQL Dockerfile:

```
FROM ubuntu  
RUN apt-get update  
RUN apt-get install -y mysql-server  
CMD ["mysqld"]
```

Ubuntu Dockerfile:

```
FROM ubuntu  
CMD ["bash"]
```

Why Ubuntu Container Exits

Problem:

```
docker run ubuntu
# Container starts and immediately exits
```

Reason: 1. Ubuntu's default CMD is `bash` 2. Bash is a shell, not a process/service 3. Bash requires a terminal to run 4. Docker doesn't attach terminal by default 5. Bash finds no terminal, exits 6. Container exits

Verification:

```
docker ps
# No running containers
```

```
docker ps -a
# Shows exited Ubuntu container
```

Overriding CMD

Method 1: Append Command to docker run

```
docker run ubuntu sleep 5
```

What Happens: - `sleep 5` replaces default `bash` command - Container runs sleep for 5 seconds - After 5 seconds, sleep exits, container stops

Method 2: Modify Dockerfile

```
FROM ubuntu
CMD ["sleep", "5"]
```

Build and Run:

```
docker build -t ubuntu-sleeper .
docker run ubuntu-sleeper
# Always sleeps for 5 seconds
```

CMD Format Options

Shell Form:

```
CMD command param1 param2
```

Example:

```
CMD sleep 5
```

Exec Form (JSON Array - Preferred):

```
CMD ["executable", "param1", "param2"]
```

Example:

```
CMD ["sleep", "5"]
```

Important Notes: - In JSON format, first element MUST be executable - Parameters as separate array elements - **Incorrect:** `dockerfile CMD ["sleep 5"]` - **Correct:** `dockerfile CMD ["sleep", "5"]`

Understanding ENTRYPOINT

Concept: ENTRYPOINT defines the executable, CMD parameters are appended to it.

Problem with CMD:

```
docker run ubuntu-sleeper 10
```

- Completely replaces `sleep 5` with `10`
- Tries to run command `10` which doesn't exist
- Error: "executable not found"

Solution - Using ENTRYPOINT:

```
FROM ubuntu
ENTRYPOINT ["sleep"]
```

Build:

```
docker build -t ubuntu-sleeper .
```

Usage:

```
docker run ubuntu-sleeper 10
```

What Happens: - ENTRYPOINT: `sleep` - Command line parameter: `10` -
Final command: `sleep 10`

Breakdown: - ENTRYPOINT cannot be overridden (by default) - Command line parameters are APPENDED to ENTRYPOINT - More intuitive for users

Combining ENTRYPOINT and CMD

Problem:

```
docker run ubuntu-sleeper
# Error: missing operand
```

Reason: No duration specified for `sleep` command.

Solution - Default Value with CMD:

```
FROM ubuntu
ENTRYPOINT ["sleep"]
CMD ["5"]
```

Behavior:

Without Parameters:

```
docker run ubuntu-sleeper
# Final command: sleep 5 (CMD value used)
```

With Parameters:

```
docker run ubuntu-sleeper 10
# Final command: sleep 10 (CMD value overridden)
```

Important Rule: - Both ENTRYPOINT and CMD must be in JSON array format - CMD values become default parameters for ENTRYPOINT - Command line arguments override CMD, not ENTRYPOINT

Overriding ENTRYPOINT

Concept: Use `--entrypoint` flag to override ENTRYPOINT at runtime.

Command:

```
docker run --entrypoint sleep2.0 ubuntu-sleeper 10
```

Breakdown: - `--entrypoint sleep2.0`: Overrides ENTRYPOINT to `sleep2.0` - `10`: Parameter passed to new ENTRYPOINT - **Final command:** `sleep2.0 10`

Use Case: Testing alternative commands without rebuilding image.

Summary Table

Scenario	Dockerfile	docker run Command	Final Command
CMD only	CMD ["sleep", "5"]	docker run ubuntu-sleeper	sleep 5
CMD override	CMD ["sleep", "5"]	docker run ubuntu-sleeper sleep 10	sleep 10
ENTRYPOINT only	ENTRYPOINT ["sleep"]	docker run ubuntu-sleeper 10	sleep 10
ENTRYPOINT + CMD	ENTRYPOINT ["sleep"]CMD ["5"]	docker run ubuntu-sleeper	sleep 5
ENTRYPOINT + CMD override	ENTRYPOINT ["sleep"]CMD ["5"]	docker run ubuntu-sleeper 10	sleep 10
Override ENTRY- POINT	ENTRYPOINT ["sleep"]	docker run --entrypoint sleep2.0 ubuntu-sleeper 10	sleep2.0 10

Docker Networking

Default Networks

Concept: Docker creates three networks automatically upon installation.

List Networks:

```
docker network ls
```

Output:

NETWORK ID	NAME	DRIVER	SCOPE
b1a2c3d4e5f6	bridge	bridge	local
a9b8c7d6e5f4	host	host	local
z9y8x7w6v5u4	none	null	local

Network Types

1. Bridge Network (Default)

Concept: Private internal network created by Docker on the host.

Characteristics: - Default network for containers - Internal IP range: Usually 172.17.x.x - Containers can communicate via internal IPs - Requires port mapping for external access

Example:

```
docker run ubuntu  
# Automatically attached to bridge network
```

Internal IP Assignment:

```
docker inspect <container_id> | grep IPAddress  
# Output: "IPAddress": "172.17.0.2"
```

Container Communication:

```
# Container 1: 172.17.0.2  
# Container 2: 172.17.0.3  
# Container 3: 172.17.0.4  
  
# Containers can ping each other using internal IPs  
docker exec container1 ping 172.17.0.3
```

External Access Requires Port Mapping:

```
docker run -p 8080:80 nginx  
# Host port 8080 mapped to container port 80
```

2. Host Network

Concept: Removes network isolation between container and host.

Command:

```
docker run --network=host nginx
```

Characteristics: - Container uses host's network directly - No port mapping needed - Container port directly accessible on host - Cannot run multiple containers on same port

Example:

```
docker run --network=host kodekloud/simple-webapp
# Application on port 5000 automatically accessible at host:5000
```

Limitations:

```
# First container works
docker run --network=host kodekloud/simple-webapp

# Second container fails (port already in use)
docker run --network=host kodekloud/simple-webapp
# Error: Address already in use
```

3. None Network

Concept: Containers are completely isolated with no network access.

Command:

```
docker run --network=none ubuntu
```

Characteristics: - No external network access - No access to other containers
- Complete network isolation - Useful for secure/isolated tasks

Creating Custom Networks

Concept: Create isolated networks for different container groups.

Command:

```
docker network create \
  --driver bridge \
  --subnet 182.18.0.0/16 \
  custom-isolated-network
```

Breakdown: - `docker network create`: Create new network - `--driver bridge`: Network type - `--subnet 182.18.0.0/16`: IP range for this network - `custom-isolated-network`: Network name

Verification:

```
docker network ls
```

Output:

NETWORK ID	NAME	DRIVER	SCOPE
b1a2c3d4e5f6	bridge	bridge	local
a9b8c7d6e5f4	host	host	local
z9y8x7w6v5u4	none	null	local
x8w7v6u5t4s3	custom-isolated-network	bridge	local

Using Custom Network:

```
docker run --network=custom-isolated-network nginx
```

Network Inspection

Command:

```
docker inspect <container_name_or_id>
```

Relevant Output Section:

```
"NetworkSettings": {
  "Bridge": "",
  "SandboxID": "a1b2c3d4e5f6...",
  "Networks": {
    "bridge": {
      "IPAddress": "172.17.0.2",
      "IPPrefixLen": 16,
      "Gateway": "172.17.0.1",
      "MacAddress": "02:42:ac:11:00:02",
      "NetworkID": "b1a2c3d4e5f6..."
    }
  },
  "Ports": {
    "80/tcp": [
      {
        "HostIp": "0.0.0.0",
        "HostPort": "8080"
      }
    ]
  }
}
```

Information Available: - Network type - Internal IP address - Gateway - MAC address - Port mappings

Container DNS Resolution

Concept: Containers can resolve each other by name using Docker's built-in DNS.

Scenario:

Web Container (172.17.0.2) ↔ MySQL Container (172.17.0.3)

Bad Practice - Using IP:

```
# In web application code  
db_connection = mysql.connect(host='172.17.0.3', port=3306)
```

Problem: IP may change on container restart.

Good Practice - Using Container Name:

```
# In web application code  
db_connection = mysql.connect(host='mysql', port=3306)
```

Docker DNS:

```
# Run MySQL with name  
docker run --name mysql mysql  
  
# Run web app linking to mysql by name  
docker run --name web-app --link mysql:mysql webapp
```

How It Works: - Docker's built-in DNS server: 127.0.0.11 - Resolves container names to their IPs - Automatically updated when containers restart

Verification:

```
# Inside web-app container  
docker exec web-app cat /etc/hosts
```

Output:

```
127.0.0.1      localhost  
172.17.0.2     mysql  
172.17.0.3     web-app
```

Network Implementation

Concept: Docker uses Linux network namespaces for container isolation.

Technical Details: - Each container gets its own network namespace - Virtual Ethernet (veth) pairs connect containers - Bridge network acts as virtual switch - iptables rules manage routing and NAT

Commands for Advanced Users:

```
# List network namespaces  
ip netns list  
  
# Execute command in namespace  
ip netns exec <namespace> <command>
```

```
# View bridge details
docker network inspect bridge
```

Docker Storage

Storage Location

Concept: Docker stores all data in a specific directory structure.

Default Storage Directory:

```
/var/lib/docker/
```

Directory Structure:

```
/var/lib/docker/
|-- aufs/           # Storage driver files (AUFS)
|-- containers/     # Container-specific files
|-- image/          # Image layers
|-- volumes/        # Persistent volumes
|-- network/        # Network configuration
-- ...
```

Purpose of Each Directory: - **aufs/**: Storage driver files (varies by storage driver) - **containers/**: Running container metadata - **image/**: Docker image layers - **volumes/**: Persistent data volumes

Layered Architecture Deep Dive

Concept: Images are built in layers; containers add a writable layer on top.

Example Dockerfile Layers:

```
FROM ubuntu          # Layer 1: Base OS (120MB)
RUN apt-get update    # Layer 2: Package index (10MB)
RUN apt-get install -y python # Layer 3: Python (300MB)
RUN pip install flask # Layer 4: Flask (45MB)
COPY app.py /opt/     # Layer 5: Source code (1KB)
ENTRYPOINT ["python", "/opt/app.py"] # Layer 6: Entrypoint (0B)
```

Layer Visualization:

```
-----
| Layer 6: ENTRYPOINT (0B) | ← Image Layer (Read-Only)
|-----|
| Layer 5: Source code (1KB) | ← Image Layer (Read-Only)
|-----|
| Layer 4: Flask (45MB) | ← Image Layer (Read-Only)
|-----|
```

Layer 3: Python (300MB)		← Image Layer (Read-Only)

Layer 2: apt update (10MB)		← Image Layer (Read-Only)

Layer 1: Ubuntu (120MB)		← Image Layer (Read-Only)

When Container Runs:

Container Layer (Read-Write)		← Writable Layer

Layer 6: ENTRYPOINT		

Layer 5: Source code		

Layer 4: Flask		← Image Layers
-----		(Read-Only, Shared)
Layer 3: Python		

Layer 2: apt update		

Layer 1: Ubuntu		

Image Layer Reuse

Concept: Multiple images share common base layers.

App1 Dockerfile:

```
FROM ubuntu
RUN apt-get update
RUN apt-get install -y python
RUN pip install flask
COPY app1.py /opt/
ENTRYPOINT ["python", "/opt/app1.py"]
```

App2 Dockerfile:

```
FROM ubuntu
RUN apt-get update
RUN apt-get install -y python
RUN pip install flask
COPY app2.py /opt/
ENTRYPOINT ["python", "/opt/app2.py"]
```

Storage Efficiency:

App1 Image:

```
|-- Layer 1-4: Shared with App2 (435MB)
|-- Layer 5: app1.py (unique, 1KB)
-- Layer 6: entrypoint (unique, 0B)
```

App2 Image:

```
|-- Layer 1-4: Shared with App1 (435MB) ← Reused, not duplicated
|-- Layer 5: app2.py (unique, 1KB)
-- Layer 6: entrypoint (unique, 0B)
```

Total Storage: 435MB + 1KB + 1KB (not 870MB)

Container Layer (Read-Write)

Concept: Each container gets its own writable layer for temporary data.

Container Layer Contents: - Log files - Temporary files - Modified files (via copy-on-write) - User-created files

Example - Creating Files:

```
docker run -it --name test-container ubuntu
```

```
# Inside container
touch /tmp/test.txt
echo "data" > /tmp/test.txt
```

File Location: - Stored in container's writable layer - Path: /var/lib/docker/containers/<container-id>/

Container Deletion:

```
docker rm test-container
```

- Writable layer deleted
- /tmp/test.txt permanently lost
- Image layers remain unchanged

Copy-on-Write Mechanism

Concept: Modifying image files creates a copy in the writable layer.

Scenario:

```
# Dockerfile includes
COPY app.py /opt/app.py
```

Image layer: /opt/app.py (read-only)

Modifying in Container:

```
docker exec -it myapp vi /opt/app.py
# Edit and save
```

What Happens: 1. Docker detects write attempt to read-only file 2. Copies /opt/app.py to writable layer 3. Modifications made to the copy 4. Original in image layer unchanged 5. Container reads from writable layer (newer version)

Layer Visualization:

Container Layer (Read-Write):
-- /opt/app.py (modified copy)

Image Layers (Read-Only):
-- /opt/app.py (original)

Multiple Containers:

```
docker run --name app1 myapp
docker run --name app2 myapp
```

Both share image layers but have separate writable layers:

```
app1 container: writable layer 1
app2 container: writable layer 2
shared: image layers (read-only)
```

Volumes - Persistent Storage

Problem: Container data is ephemeral (lost on deletion).

Solution: Mount volumes to persist data outside containers.

Creating a Volume:

```
docker volume create data_volume
```

What Happens: - Creates directory: /var/lib/docker/volumes/data_volume
- Managed by Docker - Persists independently of containers

Using the Volume:

```
docker run -v data_volume:/var/lib/mysql mysql
```

Breakdown: - -v: Mount volume - data_volume: Volume name -
/var/lib/mysql: Mount point inside container

Data Flow:

```
Container writes to /var/lib/mysql
↓
Actually stored in /var/lib/docker/volumes/data_volume
↓
Persists after container deletion
```

Auto-Creating Volumes:

```
docker run -v data_volume2:/var/lib/mysql mysql
```

- If `data_volume2` doesn't exist, Docker creates it's Equivalent to running `docker volume create data_volume2` first

Volume Mounting vs Bind Mounting

Concept: Two ways to persist data - using Docker-managed volumes or host directories.

Volume Mounting (Docker-managed):

```
docker run -v data_volume:/var/lib/mysql mysql
```

- Mounts from `/var/lib/docker/volumes/`
- Docker manages the directory
- Created with `docker volume create`

Bind Mounting (Host directory):

```
docker run -v /data/mysql:/var/lib/mysql mysql
```

- Mounts from any host directory
- Full path must be specified
- Directory can exist anywhere on host

Complete Example:

Bind mount to external storage

```
docker run -v /data/mysql:/var/lib/mysql mysql
```

Volume mount (Docker-managed)

```
docker run -v data_volume:/var/lib/mysql mysql
```

Visual Comparison:

Volume Mount:

Host: `/var/lib/docker/volumes/data_volume/` ↔ Container: `/var/lib/mysql`

Bind Mount:

Host: `/data/mysql/` ↔ Container: `/var/lib/mysql`

Modern Mount Syntax

Concept: `--mount` is the preferred syntax over `-v` for clarity.

Old Syntax:

```
docker run -v /data/mysql:/var/lib/mysql mysql
```

New Syntax:

```
docker run --mount type=bind,source=/data/mysql,target=/var/lib/mysql mysql
```


Breakdown: - `--mount`: More explicit mount syntax - `type=bind`: Mount type (bind, volume, tmpfs) - `source=/data/mysql`: Source path on host - `target=/var/lib/mysql`: Destination path in container

Volume Mount Example:

```
docker run --mount type=volume,source=data_volume,target=/var/lib/mysql mysql
```

Advantages of `--mount`: - More verbose and explicit - Better error messages
- Consistent key=value format - Easier to read

Storage Drivers

Concept: Storage drivers enable the layered architecture and manage data.

Common Storage Drivers: - AUFS (Advanced multi-layered Unification FileSystem) - BTRFS (B-tree FileSystem) - Device Mapper - Overlay - Overlay2 - ZFS

Driver Selection: - Depends on underlying OS - Ubuntu default: AUFS - CentOS/Fedora: Device Mapper or Overlay2 - Docker automatically selects best available

Viewing Storage Driver:

```
docker info | grep "Storage Driver"
```

Output:

Storage Driver: overlay2

Driver Responsibilities: - Maintaining layered architecture - Creating writable container layer - Implementing copy-on-write - Managing layer storage and retrieval

Storage Driver Location:

```
/var/lib/docker/overlay2/ (for overlay2 driver)
/var/lib/docker/aufs/      (for aufs driver)
```

Docker Compose

Introduction to Docker Compose

Concept: Docker Compose simplifies running multi-container applications using YAML configuration.

Without Docker Compose:

```
# Manual approach - running multiple commands
docker run -d --name redis redis
docker run -d --name db postgres
```

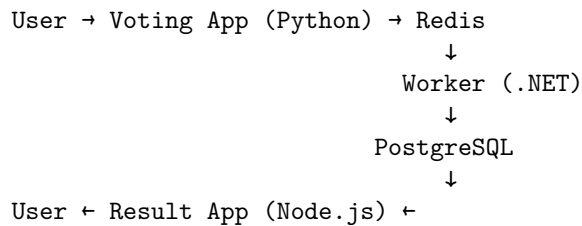
```
docker run -d --name vote -p 5000:80 voting-app
docker run -d --name result -p 5001:80 result-app
docker run -d --name worker worker
```

With Docker Compose:

```
# Single command
docker-compose up
```

Sample Voting Application

Architecture:



Component Descriptions: 1. **Voting App:** Web interface for voting (Python) 2. **Redis:** In-memory database storing votes 3. **Worker:** Processes votes from Redis to PostgreSQL (.NET) 4. **PostgreSQL:** Persistent database 5. **Result App:** Web interface showing results (Node.js)

Docker Run Commands - Manual Setup

Step 1: Start Redis

```
docker run -d --name redis redis
```

Step 2: Start PostgreSQL

```
docker run -d --name db -e POSTGRES_PASSWORD=postgres postgres
```

Step 3: Start Voting App

```
docker run -d --name vote -p 5000:80 voting-app
```

Step 4: Start Result App

```
docker run -d --name result -p 5001:80 result-app
```

Step 5: Start Worker

```
docker run -d --name worker worker
```

Problem: Containers are isolated - they can't communicate!

Linking Containers (Deprecated Method)

Concept: Use `--link` to connect containers (legacy approach).

Updated Commands:

Voting App with Redis Link:

```
docker run -d --name vote -p 5000:80 --link redis:redis voting-app
```

Breakdown: - `--link redis:redis`: Links to container named “redis” - Format: `--link <container_name>:<hostname>` - Creates entry in `/etc/hosts` file

Result App with DB Link:

```
docker run -d --name result -p 5001:80 --link db:db result-app
```

Worker with Both Links:

```
docker run -d --name worker --link redis:redis --link db:db worker
```

How Links Work:

```
# Inside voting-app container  
cat /etc/hosts
```

Output:

```
127.0.0.1      localhost  
172.17.0.2     redis  
172.17.0.3     vote
```

Important: `--link` is deprecated. Modern approach uses user-defined networks.

Docker Compose File - Version 1

Concept: Translate docker run commands into YAML format.

docker-compose.yml (Version 1):

```
redis:  
  image: redis  
  
db:  
  image: postgres  
  environment:  
    POSTGRES_PASSWORD: postgres  
  
vote:  
  image: voting-app  
  ports:  
    - 5000:80
```

```

links:
  - redis

result:
  image: result-app
  ports:
    - 5001:80
  links:
    - db

worker:
  image: worker
  links:
    - redis
    - db

```

Structure Breakdown:

Service Definition:

```

redis:                                # Service name
  image: redis                        # Image to use

```

Port Mapping:

```

vote:
  ports:
    - 5000:80                        # host_port:container_port

```

Environment Variables:

```

db:
  environment:
    POSTGRES_PASSWORD: postgres

```

Links:

```

vote:
  links:
    - redis                          # Link to redis service

```

Running the Application:

```
docker-compose up
```

What Happens: 1. Reads `docker-compose.yml` 2. Creates all containers 3. Sets up links 4. Starts containers in dependency order 5. Shows combined logs

Building Images with Docker Compose

Concept: Build images as part of docker-compose instead of using pre-built images.

Modified docker-compose.yml:

```
redis:
  image: redis

db:
  image: postgres
  environment:
    POSTGRES_PASSWORD: postgres

vote:
  build: ./vote          # Build from local directory
  ports:
    - 5000:80
  links:
    - redis

result:
  build: ./result        # Build from local directory
  ports:
    - 5001:80
  links:
    - db

worker:
  build: ./worker        # Build from local directory
  links:
    - redis
    - db
```

Directory Structure:

```
project/
|-- docker-compose.yml
|-- vote/
|   |-- Dockerfile
|   -- app.py
|-- result/
|   |-- Dockerfile
|   -- server.js
-- worker/
    |-- Dockerfile
    -- Worker.cs
```

Build Process:

docker-compose up

1. Builds images from directories

2. Tags with temporary names
3. Creates and starts containers

Docker Compose Versions

Concept: Docker Compose evolved through versions with different capabilities.

Version 1 (Original):

```
redis:
  image: redis
db:
  image: postgres
vote:
  image: voting-app
  ports:
    - 5000:80
  links:
    - redis
```

Limitations: - No version specification - No network control - Links required for communication - No startup order control

Version 2:

```
version: '2'                                # Version specification required

services:                                    # All services under 'services' key
  redis:
    image: redis

  db:
    image: postgres

  vote:
    image: voting-app
    ports:
      - 5000:80
    depends_on:                              # Startup dependencies
      - redis
```

New Features in Version 2: - Version specification at top - **services** section required - Automatic network creation (no links needed) - **depends_on** for startup order - Better isolation

Version 3 (Latest):

```
version: '3'                                # Current version

services:
```

```

redis:
  image: redis

db:
  image: postgres

vote:
  image: voting-app
  ports:
    - 5000:80
  depends_on:
    - redis

```

New Features in Version 3: - Docker Swarm support - Stack deployment -
Some options removed/added

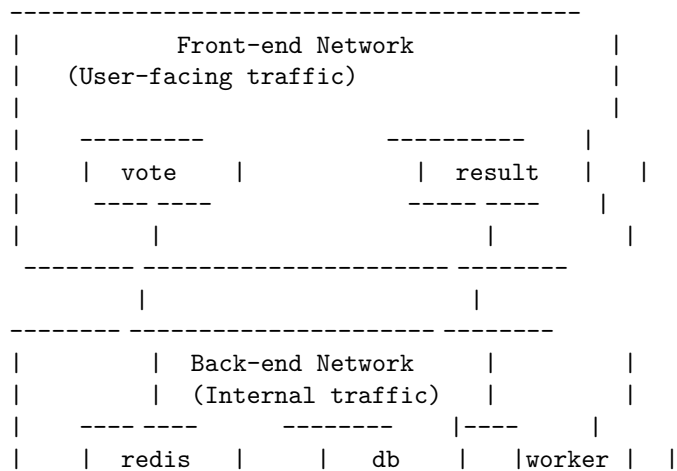
Version Comparison:

Feature	Version 1	Version 2	Version 3
Version spec	No	Yes	Yes
Services section	No	Yes	Yes
Auto networking	No	Yes	Yes
Links needed	Yes	No	No
depends_on	No	Yes	Yes
Swarm support	No	No	Yes

Networks in Docker Compose

Concept: Define custom networks to segregate traffic.

Network Architecture:



```

| ----- |
|-----|

docker-compose.yml with Networks:

version: '3'

services:
  redis:
    image: redis
    networks:
      - backend          # Only on backend network

  db:
    image: postgres
    networks:
      - backend          # Only on backend network

  vote:
    image: voting-app
    ports:
      - 5000:80
    networks:
      - frontend          # On both networks
      - backend

  result:
    image: result-app
    ports:
      - 5001:80
    networks:
      - frontend          # On both networks
      - backend

  worker:
    image: worker
    networks:
      - backend          # Only on backend network

networks:
  frontend:
  backend:

```

Network Breakdown:

Defining Networks:

```
networks:
```



```
frontend:          # Network name
backend:           # Network name
```

Assigning to Services:

```
vote:
  networks:
    - frontend      # Connected to frontend
    - backend        # Connected to backend
```

Benefits: - Isolate traffic types - Security boundaries - Organized architecture

Running with Networks:

```
docker-compose up
```

Docker automatically creates: - `project_frontend` network - `project_backend` network - Attaches services as specified

Docker Registry

Understanding Docker Registry

Concept: Docker Registry is a repository for storing and distributing Docker images.

Image Naming Convention:

```
[registry]/[username]/[repository]:[tag]
```

Examples:

```
# Full format
docker.io/nginx/nginx:latest
```

```
# Short format (defaults)
nginx
```

```
# Expands to: docker.io/library/nginx:latest
```

Component Breakdown: - `docker.io`: Registry DNS name (Docker Hub) - `nginx`: Username/Organization (library is default) - `nginx`: Repository name - `latest`: Tag/Version

Public Registries

Docker Hub (docker.io): - Default public registry - Official images from vendors - Community images

Google Container Registry (gcr.io): - Kubernetes-related images - Google Cloud images

Example URLs:

Docker Hub

```
docker pull docker.io/nginx:latest
```

Google Registry

```
docker pull gcr.io/kubernetes-helm/tiller:v2.11.0
```

Private Registries

Concept: Host images internally for proprietary applications.

Cloud Provider Registries: - AWS: Amazon Elastic Container Registry (ECR) - Azure: Azure Container Registry (ACR) - GCP: Google Container Registry (GCR)

Using Private Registry:

Step 1: Login

```
docker login private-registry.io
```

Prompts:

Username: myuser

Password: *****

Step 2: Pull/Run Image

```
docker run private-registry.io/apps/internal-app
```

Without Login:

Error response from daemon: pull access denied for private-registry.io/apps/internal-app

Deploying Private Registry

Concept: Run your own Docker registry using the official registry image.

Command:

```
docker run -d -p 5000:5000 --name registry registry:2
```

Breakdown: - registry:2: Official Docker registry image - -p 5000:5000: Exposes API on port 5000 - -d: Runs in background - --name registry: Names the container

Registry is Now Running: - Accessible at localhost:5000 - Provides REST API for push/pull operations

Pushing Images to Private Registry

Step 1: Tag Image with Registry URL

```
docker image tag my-image localhost:5000/my-image
```

Breakdown: - `docker image tag`: Tag an existing image - `my-image`: Current image name - `localhost:5000/my-image`: New name with registry URL

Step 2: Push to Registry

```
docker push localhost:5000/my-image
```

Verification:

```
# List images in registry  
curl -X GET http://localhost:5000/v2/_catalog
```

Output:

```
{  
  "repositories": ["my-image"]  
}
```

Pulling from Private Registry

From Same Host:

```
docker pull localhost:5000/my-image
```

From Another Host:

```
docker pull 192.168.1.100:5000/my-image
```

Tagging for Internal Registry:

```
# Tag local image for internal registry  
docker image tag my-app registry.company.com:5000/my-app
```

```
# Push to company registry  
docker push registry.company.com:5000/my-app
```

```
# Pull from anywhere in company network  
docker pull registry.company.com:5000/my-app
```

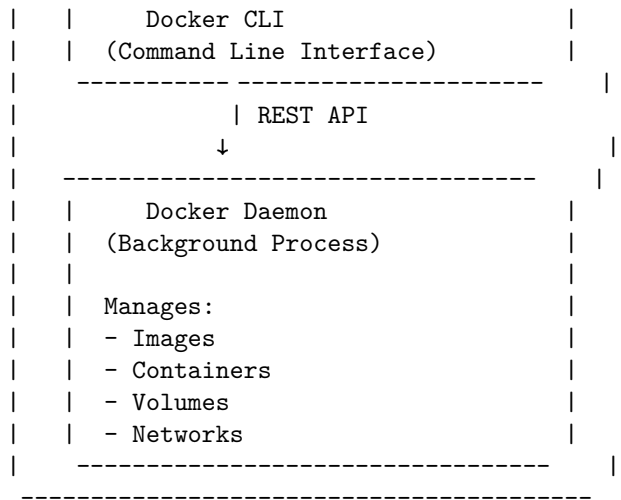
Docker Engine Architecture

Docker Components

Concept: Docker consists of three main components.

Architecture:

```
-----  
|           Docker Host           |  
|                                 |  
| -----                        |  
|                                 |
```



1. **Docker Daemon:** - Background process - Manages Docker objects - Handles container operations
2. **Docker REST API:** - Interface for communication - Used by CLI and other tools - Provides HTTP endpoints
3. **Docker CLI:** - Command-line tool (`docker` command) - Communicates with daemon via REST API - Can be remote or local

Remote Docker Engine

Concept: Docker CLI can control remote Docker engines.

Command:

```
docker -H=10.123.2.1:2375 run nginx
```

Breakdown: - `-H`: Specify remote host - `10.123.2.1`: Remote Docker host IP
 - `2375`: Docker daemon port - `run nginx`: Normal docker command

Example Use Cases:

```
# List containers on remote host
```

```
docker -H=remote-docker:2375 ps
```

```
# Run container on remote host
```

```
docker -H=remote-docker:2375 run -d redis
```

```
# Pull image on remote host
```

```
docker -H=remote-docker:2375 pull ubuntu
```

Containerization Technology

Concept: Docker uses Linux kernel features for isolation.

Linux Namespaces:

1. Process ID (PID) Namespace:

Host System:	Container:
PID 1: systemd	PID 1: nginx
PID 2: kthreadd	PID 2: app
...	...
PID 5: nginx (host view)	
PID 6: app (host view)	

How It Works: - Container process has two PIDs - PID 1 inside container (container namespace) - PID 5 on host (host namespace) - Container thinks it's the root process

Verification:

```
# On host
ps aux | grep nginx
# Output: root 5 ... nginx

# Inside container
docker exec <container> ps aux
# Output: root 1 ... nginx
```

2. Other Namespaces: - **Network:** Separate network stack - **Mount:** Separate filesystem mount points - **IPC:** Inter-process communication - **UTS:** Host-name and domain name - **User:** User and group IDs

Resource Limits (cgroups)

Concept: Control groups limit resource usage per container.

Setting CPU Limit:

```
docker run --cpus=0.5 ubuntu
```

Breakdown: - --cpus=0.5: Limit to 50% of one CPU core - Container cannot exceed this limit - Ensures fair resource sharing

Setting Memory Limit:

```
docker run --memory=100m ubuntu
```

Breakdown: - --memory=100m: Limit to 100 megabytes - Container killed if exceeds (OOM) - Prevents memory exhaustion

Combined Limits:

```
docker run --cpus=0.5 --memory=512m nginx
```

```
docker stats
```

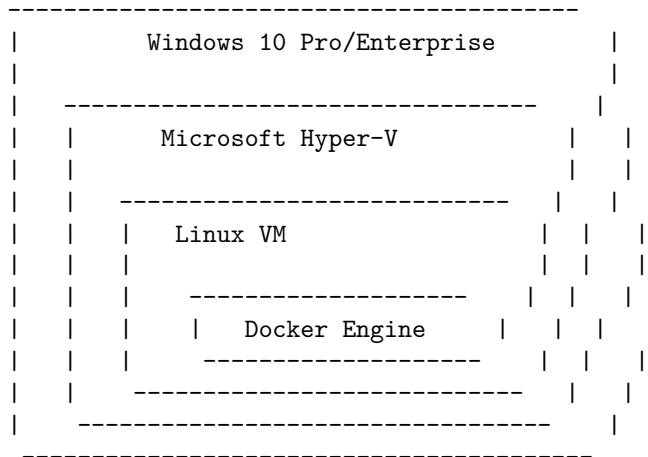
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %
a043d2346f12	webapp	0.50%	128MiB / 512MiB	25.00%
b1a2c3d4e5f6	database	1.00%	450MiB / 1GiB	43.95%

```

|                               |
|       Windows/Mac Host      |
|                               |
|-----|
|       Oracle VirtualBox     |
|                               |
|-----|
|       Boot2Docker VM        |
|       (Lightweight Linux)    |
|                               |
|       -----|
|       | Docker Engine        |
|       | (Linux Containers)   |
|       |-----|
|       -----|
|-----|

```

Architecture:



Requirements: - Windows 10 Pro, Enterprise, or Education - Windows Server 2016 or higher - Hyper-V support (enabled) - 64-bit processor with SLAT

Advantages: - Native integration - Better performance - Native filesystem access

Note: VirtualBox and Hyper-V cannot coexist.

Windows Containers

Concept: Native Windows containers on Windows hosts.

Types:

1. Windows Server Containers:

```
docker run microsoft/windowsservercore
```

- Shares kernel with host
- Similar to Linux containers
- Lightweight isolation

2. Hyper-V Containers:

```
docker run --isolation=hyperv microsoft/nanoserver
```

- Each container in own VM
- Complete kernel isolation
- Better security boundaries

Base Images:

Windows Server Core:

FROM microsoft/windowsservercore

- Full Windows Server features

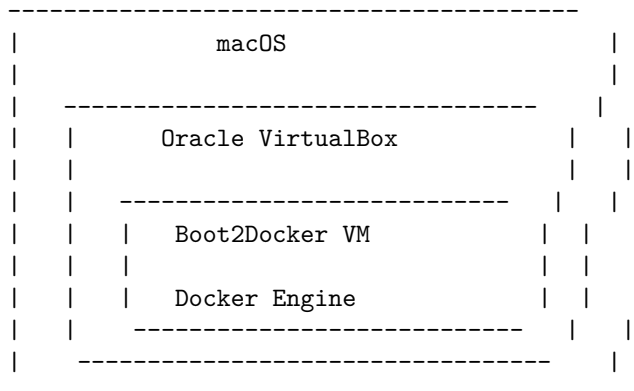
- ### Nano Server:

- Minimal Windows Server
- Smaller size (~400MB)
- Headless deployment
- Similar to Alpine Linux

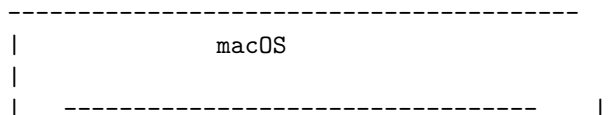
```
docker version
```

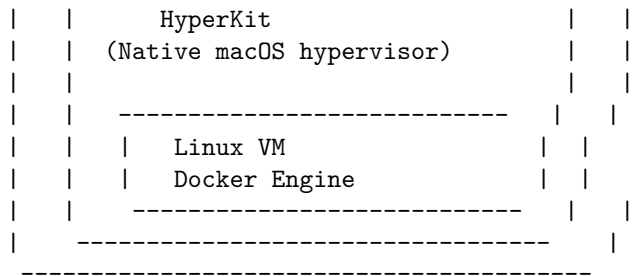
```
docker run microsoft/nanoserver cmd /c echo "Hello Windows"
```

Architecture (Docker Toolbox):



Docker Desktop for Mac (Modern):





Requirements: - macOS Sierra 10.12 or newer - 2010 or newer Mac hardware

Note: - No native Mac containers exist - All containers are Linux-based - Uses HyperKit (native virtualization)

Container Orchestration

Why Container Orchestration?

Concept: Automated management of containerized applications at scale.

Problems Without Orchestration:

1. Manual Scaling:

Running one instance

```
docker run nodejs-app
```

Need more? Run manually

```
docker run nodejs-app
```

```
docker run nodejs-app
```

2. No Health Monitoring: - If container fails, must manually restart - No automatic failover

3. No Load Balancing: - Manual port mapping - Manual configuration

4. Manual Resource Management: - No automatic host selection - No resource optimization

Container Orchestration Solutions

Concept: Tools that automate deployment, scaling, and management.

Features: - Automatic scaling - Health monitoring and self-healing - Load balancing - Service discovery - Rolling updates - Resource optimization - Multi-host networking

Popular Solutions:

Output:

This node joined a swarm as a worker.

Docker Services

Concept: Services manage containerized applications in swarm mode.

Creating a Service:

```
docker service create --replicas=3 --name web-app nodejs-app
```

Breakdown: - `docker service create`: Create new service - `--replicas=3`: Run 3 instances - `--name web-app`: Service name - `nodejs-app`: Image to use

What Happens: 1. Swarm manager schedules 3 containers 2. Distributes across worker nodes 3. Monitors health 4. Maintains desired state (3 replicas)

Listing Services:

```
docker service ls
```

Output:

ID	NAME	MODE	REPLICAS	IMAGE
abc123def456	web-app	replicated	3/3	nodejs-app:latest

Listing Service Tasks (Containers):

```
docker service ps web-app
```

Output:

ID	NAME	NODE	STATE	IMAGE
xyz1	web-app.1	worker1	Running	nodejs-app
xyz2	web-app.2	worker2	Running	nodejs-app
xyz3	web-app.3	worker3	Running	nodejs-app

Scaling a Service:

```
docker service scale web-app=5
```

Auto-Healing: - If container fails, swarm restarts it - If node fails, swarm reschedules containers

Kubernetes Introduction

What is Kubernetes?

Concept: Open-source container orchestration platform for automating deployment, scaling, and management.

Key Features: - Automatic scaling (up/down) - Self-healing - Load balancing - Rolling updates/rollbacks - Secret management - Service discovery - Storage orchestration

Comparison to Docker Run:

Docker:

```
docker run nodejs-app
```

- Runs single instance
- Manual scaling
- No auto-restart

Kubernetes:

```
kubectl run --replicas=1000 nodejs-app
```

- Runs 1000 instances
- Automatic distribution
- Self-healing

Scaling:

```
kubectl scale --replicas=2000 nodejs-app
```

- Instant scaling to 2000 instances
- Automatic across nodes

Rolling Update:

```
kubectl rolling-update nodejs-app --image=nodejs-app:v2
```

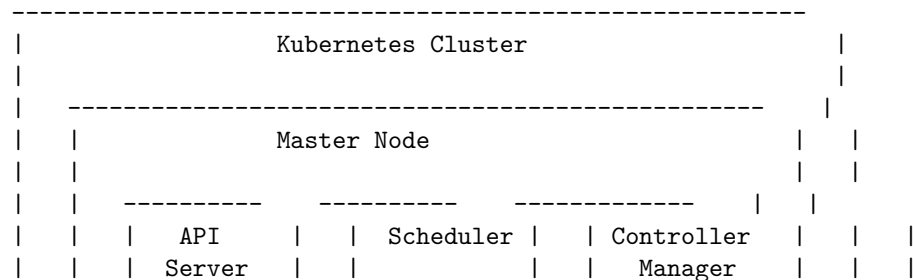
- Updates one instance at a time
- Zero downtime

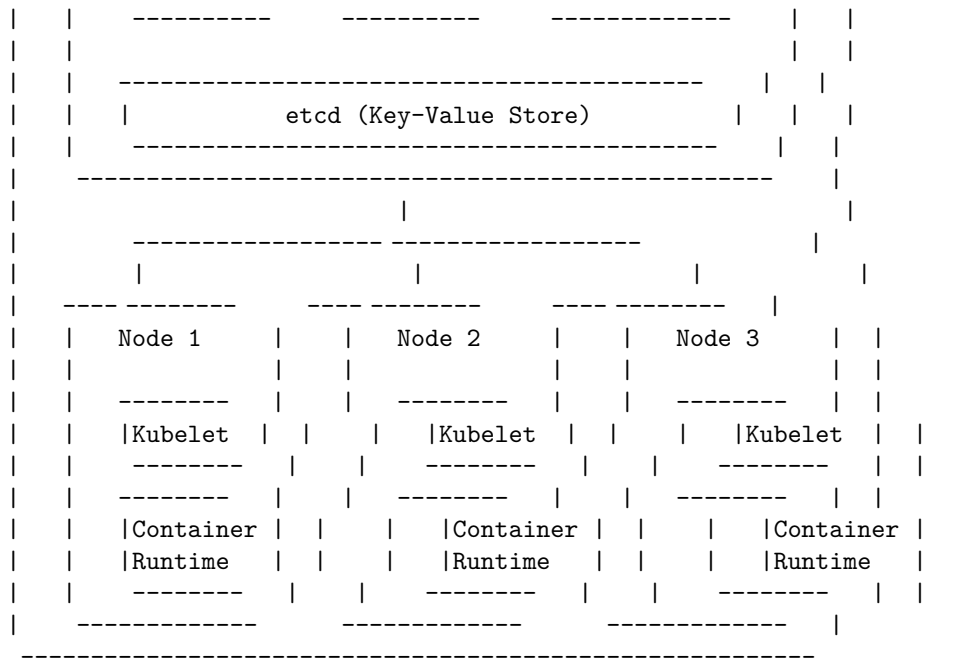
Rollback:

```
kubectl rollback nodejs-app
```

- Reverts to previous version
- Automatic

Kubernetes Architecture





Master Node Components:

1. **API Server:** - Front-end for Kubernetes - All communication goes through it - RESTful API
2. **etcd:** - Distributed key-value store - Stores all cluster data - Maintains cluster state
3. **Scheduler:** - Assigns pods to nodes - Considers resource requirements - Respects constraints
4. **Controller Manager:** - Maintains desired state - Node Controller - Replication Controller - Endpoints Controller

Worker Node Components:

1. **Kubelet:** - Agent running on each node - Communicates with API server - Manages pods and containers
2. **Container Runtime:** - Runs containers (Docker, containerd, CRI-O) - Pulls images - Starts/stops containers
3. **Kube-proxy:** - Network proxy - Load balancing - Service discovery

Basic Kubernetes Commands

Running an Application:

```
kubectl run my-app --image=my-app --replicas=100
```

Breakdown: - `kubectl run`: Create deployment - `my-app`: Deployment name
- `--image=my-app`: Container image - `--replicas=100`: Number of instances

Scaling:

```
kubectl scale --replicas=200 deployment/my-app
```

Viewing Cluster:

```
# Cluster info  
kubectl cluster-info
```

```
# List nodes  
kubectl get nodes
```

```
# List pods  
kubectl get pods
```

```
# List services  
kubectl get services
```

Course Conclusion

Key Takeaways

What You've Learned:

1. **Docker Basics:**
 - What containers are
 - How Docker works
 - Container vs VM
2. **Docker Commands:**
 - Running containers
 - Managing images
 - Networking and storage
3. **Building Images:**
 - Dockerfile creation
 - Layered architecture
 - Best practices
4. **Docker Compose:**
 - Multi-container applications
 - YAML configuration
 - Service orchestration
5. **Advanced Concepts:**
 - Docker storage
 - Networking
 - Registry management

6. Orchestration:

- Docker Swarm basics
- Kubernetes introduction
- Production considerations

Next Steps

Further Learning: - Docker Swarm advanced course - Kubernetes for beginners - Kubernetes certification - Ansible, Chef, Puppet - CI/CD with Docker

Resources: - KodeKloud (kodekloud.com) - Docker Documentation (docs.docker.com) - Kubernetes Documentation (kubernetes.io) - Practice labs and hands-on exercises

Complete Command Reference

Installation Commands

```
# Check Ubuntu version
cat /etc/lsb-release

# Remove old Docker versions
sudo apt-get remove docker docker-engine docker.io containerd runc

# Install via convenience script
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Verify installation
docker version
```

Container Lifecycle Commands

```
# Run container
docker run <image>
docker run -d <image> # Detached mode
docker run -it <image> # Interactive mode
docker run --name <name> <image> # Named container
docker run -p <host>:<container> <image> # Port mapping
docker run -v <host>:<container> <image> # Volume mapping
docker run -e KEY=VALUE <image> # Environment variable

# List containers
docker ps # Running containers
docker ps -a # All containers
```

```

# Stop containers
docker stop <container>

# Start stopped container
docker start <container>

# Remove container
docker rm <container>

# Execute command in container
docker exec <container> <command>
docker exec -it <container> bash           # Interactive shell

# View logs
docker logs <container>

# Inspect container
docker inspect <container>

# Attach to container
docker attach <container>

```

Image Management Commands

```

# List images
docker images

# Pull image
docker pull <image>:<tag>

# Build image
docker build -t <name>:<tag> .
docker build -t <name> -f <dockerfile> .

# Tag image
docker tag <source> <target>

# Push image
docker push <image>:<tag>

# Remove image
docker rmi <image>

# View image history
docker history <image>

```


Remove unused images

`docker image prune`

Network Commands

List networks

`docker network ls`

Create network

`docker network create <name>`

`docker network create --driver bridge --subnet 182.18.0.0/16 <name>`

Inspect network

`docker network inspect <network>`

Connect container to network

`docker network connect <network> <container>`

Disconnect container from network

`docker network disconnect <network> <container>`

Volume Commands

Create volume

`docker volume create <name>`

List volumes

`docker volume ls`

Inspect volume

`docker volume inspect <volume>`

Remove volume

`docker volume rm <volume>`

Remove unused volumes

`docker volume prune`

Docker Compose Commands

Start services

`docker-compose up`

`docker-compose up -d`

Detached mode

Stop services

`docker-compose down`

```
# List services
docker-compose ps

# View logs
docker-compose logs
docker-compose logs <service>

# Scale service
docker-compose scale <service>=<count>

# Build images
docker-compose build
```

Docker Swarm Commands

```
# Initialize swarm
docker swarm init --advertise-addr <ip>

# Join swarm as worker
docker swarm join --token <token> <manager-ip>:2377

# List nodes
docker node ls

# Create service
docker service create --name <name> --replicas <count> <image>

# List services
docker service ls

# List service tasks
docker service ps <service>

# Scale service
docker service scale <service>=<count>

# Update service
docker service update --image <image>:<tag> <service>

# Remove service
docker service rm <service>
```

System Commands

```
# View system info
docker info
```

```
# View disk usage
docker system df

# Remove all unused objects
docker system prune

# View resource usage
docker stats

# View events
docker events
```